

1. Plan de Testing:

A. Alcance de las Pruebas:

El alcance se centra en la lógica algorítmica y de negocio del controlador `authController.js`. Tras revisar la implementación, las pruebas validarán de forma estricta:

- En Registro (`/api/registro`): Integridad en la recepción de 5 parámetros (nombre, username, email, password, password2), validación de coincidencia de contraseñas, políticas de longitud estricta (≥ 8 caracteres), e interceptación de colisiones de datos redundantes duplicados en base de datos PostgreSQL.
- En Autenticación (`/api/login`): Consultas indexadas relacionales (JOIN entre tablas USUARIO y ROL), verificación de flujo híbrido de contraseñas (comparación directa para credenciales heredadas en plano y validación asíncrona mediante `bcrypt.compare` para contraseñas hasheadas), y expedición de firmas criptográficas de tokens de sesión JWT (JSON Web Tokens) con expiración temporal de 2 horas.

Nota técnica: Las funcionalidades correspondientes al catálogo extendido de videojuegos, comentarios de la comunidad y agenda de eventos quedan fuera del alcance de este ciclo de pruebas, al encontrarse pendientes de implementación.

B. Tipos de Prueba Utilizados:

Para asegurar la robustez, seguridad e integridad de la arquitectura del backend, se han combinado tres niveles estratégicos de pruebas de software:

- Pruebas Unitarias Lógicas (Unit Tests): Enfocadas en verificar el comportamiento de las funciones registro y login de forma aislada dentro de `authController.js`. Estas pruebas validan de forma estricta los bloques condicionales del código antes de interactuar con la base de datos. Se centran en comprobar que el cortocircuito de la petición funcione ante datos inválidos, forzando las respuestas esperadas:
 - Falta de parámetros obligatorios (codigo 400).
 - Discordancia de claves en el registro (codigo 400).
 - Validación de longitud mínima de la contraseña (codigo 400).
 - Verificación matemática de los hashes criptograficos mediante `bcrypt.compare` (codigo 401).

- **Pruebas de Integración de Base de Datos y Servicios:** Destinadas a validar el flujo de comunicación síncrona y asíncrona End-to-End entre las rutas expuestas por Express en app.js, la lógica de control y los servicios externos. Específicamente, comprueban:
 - La correcta ejecución de las consultas parametrizadas contra el motor PostgreSQL mediante pool.query.
 - La interpretación lógica cuando la base de datos detecta duplicados en el índice único de email o username (codigo 400).
- **Pruebas de Caja Negra (Black-Box Testing / API Testing):** Realizadas inyectando payloads en formato JSON directamente en los puntos de entrada (endpoints) /api/registro y /api/login mediante Thunder Client, sin alterar una sola línea del código fuente. Estas pruebas simulan el comportamiento exacto que tendrán las peticiones asíncronas enviadas desde los formularios del frontend, evaluando el comportamiento del sistema basándose únicamente en las entradas proporcionadas y las respuestas HTTP resultantes.

C. Criterios de Aceptación General:

Para dar por válido cada escenario de prueba del sistema, se deben cumplir rigurosamente los siguientes criterios:

- **Consistencia del Estado HTTP:** El servidor debe responder con el código de estado estándar adecuado para cada acción (v.g., 201 Created para registros exitosos, 200 OK para logins correctos, 400 Bad Request para errores de validación de usuario o datos duplicados, y 401 Unauthorized para fallos de credenciales).
- **Integridad de las Respuestas:** Toda respuesta del servidor ante un escenario de fallo debe retornar un objeto estructurado en formato JSON que contenga una clave explícita "error", detallando el motivo de la interrupción (por ejemplo: {"error": "Todos los campos son obligatorios."}).
- **Seguridad y Persistencia:** Ningún usuario con datos incompletos, contraseñas inválidas o correos duplicados debe llegar a registrarse o alterar las tablas del sistema de persistencia.

D. Herramientas de Testing Utilizadas:

Para la ejecución de este plan de pruebas, se ha prescindido de entornos gráficos pesados externos, optando por herramientas de ecosistema integrado que agilizan el ciclo de desarrollo:

- Node.js & Express Framework: Entorno de ejecución y servidor de desarrollo sobre el cual se levanta la API local en el puerto estipulado (por defecto, `http://localhost:3000`), encargado de procesar las peticiones y ejecutar la lógica de control bajo prueba.
- Thunder Client (Extensión de VS Code): Cliente API REST integrado directamente en el entorno de desarrollo para el diseño, envío y monitorización de peticiones HTTP (POST) con payloads en formato JSON hacia los endpoints de la API.
- Sistema de Gestión de Bases de Datos (PostgreSQL): Utilizado como herramienta de soporte para la ejecución de consultas de verificación directa, asegurando que los datos aceptados por las pruebas se consolidan correctamente y que las restricciones de integridad relacional se cumplen en el motor de persistencia.

2. Juegos de Prueba:

ID	Descripción	Datos de entrada	Resultado esperado	Resultado real
CP-01	Registro exitoso de un nuevo usuario en el sistema con parámetros válidos.	{ "nombre": "Carlos Mendoza", "username": "carlos_gamer26", "email": "carlos@gamehub.com", "password": "PasswordSegura123" , "password2":	Status: 201 Created { "mensaje": "Bienvenido/a, Carlos Mendoza. Cuenta creada.", "token": "eyJhbGciOiJIUzI1NiIsInR5cGU6Ij	Status: 201 Created { "mensaje": "Bienvenido/a, Carlos Mendoza. Cuenta creada.", "token": "eyJhbGciOiJIUzI1NiIsInR5cGU6Ij

		<pre>"PasswordSegura123" }</pre>	<pre>CI6lKpXVCJ9.e yJpZCI6InVzcl 84N2FmNjk5N CIsInJvbCI6IIN 1c2NyaXB0b3Ii LCJpYXQiOjE 3NzkzMDE1Nz YsImV4cCI6M Tc3OTMwODc 3Nn0.ELQH5Ej INq0h5_qr7cpv K8Qmw1JSy-T 8dc2EioOk20g" ' "rol": "Suscriptor" }</pre>	<pre>CI6lKpXVCJ9.e yJpZCI6InVzcl 84N2FmNjk5N CIsInJvbCI6IIN 1c2NyaXB0b3Ii LCJpYXQiOjE 3NzkzMDE1Nz YsImV4cCI6M Tc3OTMwODc 3Nn0.ELQH5Ej INq0h5_qr7cpv K8Qmw1JSy-T 8dc2EioOk20g" ' "rol": "Suscriptor" } Estado: PASSED</pre>
CP-02	Registro fallido cuando las contraseñas introducidas no coinciden entre sí.	<pre>{ "nombre": "Elena R.", "username": "elena_dev", "email": "elena@test.com", "password": "Password123", "password2": "OTRA_CLAVE" }</pre>	<pre>Status: 400 Bad Request { "error": "Las contraseñas no coinciden." }</pre>	<pre>Status: 400 Bad Request { "error": "Las contraseñas no coinciden." } Estado: PASSED</pre>
CP-03	Registro fallido si la longitud de la contraseña es menor a 8 caracteres.	<pre>{ "nombre": "Elena R.", "username": "elena_dev", "email": "elena@test.com", "password": "123", "password2": "123" }</pre>	<pre>Status: 400 Bad Request { "error": "La contraseña debe tener al menos 8 caracteres." }</pre>	<pre>Status: 400 Bad Request { "error": "La contraseña debe tener al menos 8 caracteres." } Estado: PASSED</pre>

			1Rr9a6o", "rol": "Suscriptor" }	1Rr9a6o", "rol": "Suscriptor" } Estado: PASSED
CP-06	Inicio de sesión fallido cuando el correo electrónico no está registrado en la base de datos.	{ "email": "no_existo_2026@gamehub.com", "password": "PasswordSegura123" }	Status: 404 Not Found { "error": "Usuario no encontrado" }	Status: 404 Not Found { "error": "Usuario no encontrado" } Estado: PASSED
CP-07	Inicio de sesión fallido cuando el usuario existe pero la contraseña introducida es incorrecta.	{ "email": "carlos@gamehub.com", "password": "CLAVE_ERRONEA_99" }	Status: 401 Unauthorized { "error": "Contraseña incorrecta" }	Status: 401 Unauthorized { "error": "Contraseña incorrecta" } Estado: PASSED